Ders 11 Çalışma Özeti

Ders 11 Çalışma Özeti

Genel Konular

- Yığın (stack) ve kuyruk (queue) karşılaştırması: LIFO ve FIFO.
- Dizi ile kuyruk: kaydırmalı kuyruk ve dairesel kuyruk (circular queue).
- Bağlı liste (linked list) ile kuyruk: `QNode` ve `Queue` yapıları, `createQueue`, `newNode`, `enQueue`, `deQueue`.
- Ağaç (tree) veri yapısı: kullanım alanları ve terminoloji.
- Düğüm derecesi (degree), seviye (level), derinlik (depth), yükseklik (height).
- Kenar (edge), yol (path), alt ağaç (subtree), orman (forest).
- Ağacın dizi ve bağlı liste ile gösterimi.
- İkili ağaç (binary tree) ve dizide `2i+1` / `2i+2` yerleşimi.
- Ağaçta gezinme (traversal): preorder, inorder, postorder.
- Özyinelemeli (recursive) `printPreorder` fonksiyonu.

Hocanın Özellikle Vurguladığı Kısımlar

- Kuyruk boyutu tahmin edilebiliyorsa dairesel dizi kullanılmalıdır. Boyut hiç kestirilemiyorsa bağlı liste kullanılmalıdır.
- Bağlı listenin maliyeti vardır. Her eleman ek bir adres tutar ve her eklemede bellek ayırma (allocation) yapılır.
- `enQueue` öncesinde kuyruğun dolu olup olmadığı (`isFull`), `deQueue` öncesinde boş olup olmadığı (`isEmpty`) kontrol edilmelidir.
- `deQueue` işleminde son eleman çıkınca `rear` da `NULL` yapılmalıdır. Aksi hâlde `rear` serbest bırakılmış bir adresi gösterir.
- Kuyruktan çıkan düğüm `free` ile serbest bırakılmalıdır. Program hata vermez ama bellek boşa tutulur.
- `malloc` sonucu mutlaka kontrol edilmelidir. Hoca derste vakit kazanmak için bu kontrolü yazmadı.
- Kuyruk tüm program tarafından kullanılmıyorsa global tanımlanmamalıdır. Parametre olarak aktarılmalıdır.
- `front` ve `rear` sıradan değişken değil, işaretçidir (pointer). Tipleri `QNode *` olmalıdır.
- Ağaçta iki düğüm arasında tek bir yol vardır. Bir düğümün birden fazla ebeveyni (parent) olamaz.
- Ağaç ve graf en çok kullanılan veri yapılarıdır. Bu yüzden bağlı listeyi iyi öğrenmek gerekir.
- Öğrencilerden `isEmpty`, dairesel kuyruk algoritması, `printInorder` ve `printPostorder` fonksiyonlarını kendilerinin yazması istendi.

Kısa Tekrar Notları

- Yığın: son giren ilk çıkar (Last In First Out). Örnek: metin editöründe undo/redo, tarayıcıda geri gitme, fonksiyon çağrıları.
- Kuyruk: ilk giren ilk çıkar (First In First Out). Örnek: market kuyruğu.
- Kuyrukta iki işaretçi vardır: `front` kuyruğun başını, `rear` kuyruğun sonunu gösterir.
- Kaydırmalı kuyrukta her çıkışta en kötü durumda $n - 1$ eleman kaydırılır.
- Dairesel kuyrukta yalnızca `front` ve `rear` hareket eder. Elemanlar yerinden oynamaz.
- `peek`: kuyruğun başındaki elemanı çıkarmadan döndürür.

- Kök (root): ebeveyni olmayan tek düğüm.
- Yaprak (leaf): çocuğu olmayan düğüm.
- İç düğüm (internal node): en az bir çocuğu olan düğüm. Ebeveyn düğüm (parent node) ile aynı anlamdadır.
- Kardeş düğümler (sibling): aynı ebeveyne bağlı düğümler.
- Derece: bir düğümün çocuk sayısı.
- Seviye ve derinlik: düğümün kökten uzaklığı. Kök seviye 0'dır.
- Yükseklik: düğümden en uzak yaprağa olan uzaklık. Yaprakların yüksekliği 0'dır.
- Ağacın yüksekliği kökün yüksekliğine eşittir.
- İkili ağaçta her düğümün en fazla iki çocuğu vardır.
- Dizide i. düğümün sol çocuğu $2i + 1$, sağ çocuğu $2i + 2$ indisindedir.
- Preorder: düğüm, sol, sağ. Inorder: sol, düğüm, sağ. Postorder: sol, sağ, düğüm.

Detaylı Açıklamalar

Yığın ve Kuyruk Karşılaştırması

- Yığında amaç son işlem yapılan elemana ilk erişmektir.
- Hoca iPad'de metin yazıp undo yaparak bunu gösterdi. Undo en son yazılanı siler. Redo en son silineni geri getirir.
- Undo/redo aslında iki yığın kullanır. Birinden alınan diğerine konur.
- Kuyrukta ilk gelene ilk hizmet verilir. Yeni gelen kuyruğun sonuna eklenir.

Dizi ile Kuyruk

- **Kaydırmalı yöntem:**
  - Dizide `front` her zaman 0. gözdür.
  - Baştaki eleman çıkınca bütün elemanlar bir sola kaydırılır. `rear` bir azalır.
  - Kaydırma yapılmazsa dizinin sonunda yer kalmaz.
  - Dizi doluyken bir eleman çıkarsa $n-1$ eleman kaydırılır. Hareketli bir sistemde (market simülasyonu) bu çok masraflıdır.
- **Dairesel kuyruk:**
  - Elemanlar değil, yalnızca `front` ve `rear` işaretçileri hareket eder.
  - Örnek: A çıkınca `front` B olur. Sonra G gelirse boşalan 0. göze yerleşir.
  - `rear` son dolu gözü ya da ilk boş gözü gösterebilir. Bu bir tasarım seçimidir. Önemli olan algoritmanın tutarlı olmasıdır.
  - Dairesel kuyruk algoritmasını yazmak öğrencilere bırakıldı.
- **Hangisi seçilmeli?**
  - Boyut tahmin edilebiliyorsa (örneğin bir kasada en fazla 50 müşteri) dizi kullanılır.
  - Eleman sayısı bazen 3, bazen 300 oluyorsa bağlı liste kullanılır.

Kuyruk İşlemleri

- `enQueue`: kuyruğun sonuna eleman ekler. Diziyle gerçekleştirmede önce `isFull` kontrol edilir.
- `deQueue`: kuyruğun başındaki elemanı çıkarır. Önce `isEmpty` kontrol edilir.
- Boşluk kontrolü yalnızca hata için değildir. "Kuyruk boşalana kadar hizmet et" gibi bir tasarımda da kullanılır.
- `peek`: kimseyi yerinden oynatmadan baştaki elemanı gösterir.

Bağlı Liste ile Kuyruk

- Düğümler: A, B'yi gösterir. B, C'yi gösterir. C `NULL` gösterir. `front` A'dır, `rear` C'dir.
- `enQueue(D)`: C artık `NULL` yerine D'yi gösterir. D yeni `rear` olur ve `NULL` gösterir.
- `deQueue`: A çıkar, yeni `front` B olur.
- Düğümde istenen her bilgi tutulabilir (ad, soyad, adres). Derste basitlik için tek bir `int key` kullanıldı.

Yapı tanımları (ekrandan, 00:22:30; `typedef` adlarının eksikliği bir öğrencinin uyarısıyla düzeltildi):

```c
typedef struct QNode {
    int key;
    struct QNode *next;
} QNode;

typedef struct Queue {
    QNode *front, *rear;
} Queue;
```

- Bir öğrenci `front` ve `rear` için düz değişken kullanılıp kullanılamayacağını sordu.
- Hoca, bunların düğüm adresi tuttuğunu söyledi. Bu yüzden tipleri `QNode *` olmalıdır.

`createQueue` ve `enQueue` (ekrandan, 00:33:45):

```c
Queue *createQueue()
{
    Queue *q = (Queue *)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}

void enQueue(Queue *q, int key)
{
    QNode *tmp = newNode(key);
    if (isEmpty(q))
    {
        q->front = q->rear = tmp;
    }
    else
    {
        q->rear->next = tmp;
        q->rear = tmp;
    }
}
```

- `createQueue` bir başlatma (initialization) işlemidir. Boş kuyrukta `front` ve `rear` `NULL` olur.
- `return q;` satırı başta unutulmuştu. Bir öğrencinin uyarısıyla eklendi.
- Kuyruk boşsa yeni düğüm hem `front` hem `rear` olur.
- Kuyruk boş değilse yeni düğüm `rear`'ın arkasına bağlanır. Sonra `rear` yeni düğüm olur.

`newNode` (ekrandan, 01:14:50):

```c
QNode *newNode(int key)
{
    QNode *tmp = (QNode *)malloc(sizeof(QNode));
    tmp->key = key;
    tmp->next = NULL;
    return tmp;
}
```

- Yeni düğüm kuyruğun sonuna gireceği için `next` alanı `NULL` olur.
- Fonksiyon ayrılan yerin adresini döndürür. Dönüş tipi `QNode *`'dır.

`deQueue` (ekrandan, 00:56:50):

```c
int deQueue(Queue *q)
{
    int key;
    QNode *tmp = q->front;
    if (isEmpty(q))
    {
        printf("Queue is empty\n");
        return -1;
    }
```

```
    else
    {
        key = q->front->key;
        q->front = q->front->next;
        free(tmp);
        if (q->front == NULL)
            q->rear = NULL;
        return key;
    }
}
```

- Derste fonksiyon önce değer döndürmüyordu. İki öğrencinin uyarısıyla düzeltildi:
  - Çıkan elemanın değeri (`key`) döndürülmeli.
  - Eski `front` düğümü `free` etmek için önce `tmp`'de saklanmalı.
- `tmp` başta yanlışlıkla `Queue` tipinde yazılmıştı. Doğrusu `QNode *`'dır.
- Hata durumunda `-1` döner. Bu, `key` değerlerinin negatif olmadığı varsayımına dayanır. Algoritmaya göre farklı bir değer seçilebilir.
- Adresli örnek: A 1000, B 1500 adresinde olsun. `q->front = q->front->next` sonrası `front` 1500'ü gösterir. 1000 adresi `free` edilir.
- **Tek eleman durumu:**
  - Kuyrukta yalnızca A varsa `front` ve `rear` aynı düğümü gösterir.
  - A çıkınca `front` `NULL` olur. Ama `rear` hâlâ serbest bırakılan adresi gösterir.
  - Bu yüzden `front` `NULL` olduysa `rear` da `NULL` yapılır.
  - İki veya daha fazla eleman varken `rear` ile ilgili bir işlem gerekmez.
- `isEmpty` derste yazılmadı, öğrencilere bırakıldı. Hocanın tarifi: `front` ve `rear` ikisi de `NULL` ise kuyruk boştur.

Kullanım örneği (derste anlatılan):

```
Queue *q = createQueue();
enQueue(q, 10);
enQueue(q, 20);
printf("%d\n", deQueue(q));   /* 10 */
```

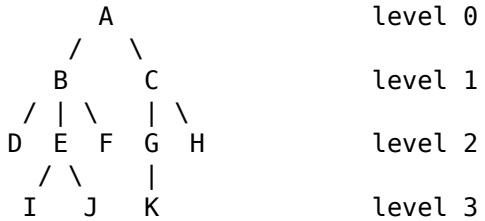
- İlk `deQueue` 10 döndürür. İkinci `deQueue` 20 döndürür.

Ağaç Veri Yapısı

- Yığın, kuyruk ve bağlı liste doğrusal (linear) yapılardır. Bilgilere belli bir sırayla erişilir.
- Ağaç doğrusal olmayan, hiyerarşik bir yapıdır.
- Gerçek dünyadaki ağacın ters çevrilmiş hâli gibi düşünülür. Kök yukarıda, yapraklar aşağıdadır.
- **Kullanım alanları:**
  - Aile ağacı.
  - Bilgisayardaki dizin (directory) yapısı: kök, altında dersler ve oyunlar, derslerin altında notlar ve ödevler.
  - Karar ağaçları (decision tree): yapay zekâda çok kullanılan bir öğrenme yöntemidir.
  - Satranç gibi oyunlarda hamlelere göre açılım.
  - Şirket organizasyon şemaları.

Ağaç Terminolojisi

Derste kullanılan örnek ağaç (ekrandan, 01:34:25):



- Ağaçta 11 düğüm vardır.
- Kök: A.
- Yapraklar: D, F, H, I, J, K.
- İç düğümler: A, B, C, E, G. Bunlar aynı zamanda ebeveyn düğümdür.
- Çocuk düğümler (child): B ve C, A'nın çocuklarıdır. D, E, F, B'nin çocuklarıdır.
- Kardeşler: B ve C kardeştir. D, E ve F kardeştir.
- Derece: B'nin derecesi 3, G'nin 1, F'nin 0'dır.
- **Seviye ve derinlik:** düğümün köke uzaklığıdır. Kökten B'ye bir adımda, K'ye üç adımda ulaşılır.
- **Yükseklik:** düğümden en alttaki yaprağa uzaklıktır.

Düğüm	Yükseklik
A	3
B, C	2
E, G	1
D, F, H, I, J, K	0

- Ağacın yüksekliği kökün yüksekliğidir, yani 3. Ağacın derinliği de 3'tür.
- Bir öğrenci farklı seviyelerdeki yaprakların hepsinin yüksekliğinin 0 olmasının karışıklık yaratıp yaratmadığını sordu.
  - Hoca karışıklık olmadığını söyledi.
  - Kökten bir düğüme kaç adımda gidileceği soruluyorsa derinlik kullanılır.
  - En uzak noktaya kaç adımda gidileceği soruluyorsa yükseklik kullanılır.
- Bir öğrenci yüksekliğin her düğümde saklanmasını önerdi. Hoca bunun algoritmanın ihtiyacına bağlı olduğunu söyledi. Sürekli kullanılıyorsa saklanır.
- **Kenar (edge):** iki düğümü bağlayan bağlantıdır. Anlamlı bilgi taşıyabilir.
  - Karar ağacında "renk kırmızı mı?" sorusunun cevabı bir kenar olabilir.
  - İstanbul ile Ankara düğümleri arasındaki kenar 1500 km gibi bir mesafe taşıyabilir.
- **Yol (path):** bir düğümden diğerine giden düğüm dizisidir. Örnek: A'dan E'ye yol A, B, E'dir.
- **Alt ağaç (subtree):** bir düğümün çocuklarından itibaren oluşan ağaçtır. A'nın iki alt ağacı vardır: B kökü ve C kökü.
- **Orman (forest):** aynı amaç için kullanılan, birbirine bağlı olmayan ağaçlar topluluğudur. Örnek: aynı apartmandaki farklı ailelerin aile ağaçları.
- Hoca terminolojinin ezberlenmesi gerekmediğini, kullandıkça öğrenileceğini söyledi. En çok kök, yaprak ve yol kullanılır.

Ağacın Bellekte Gösterimi

- Bilgisayarda hazır bir ağaç tipi yoktur. Yığın ve kuyruk gibi soyut bir yapıdır, programcı kurar.
- **Dizi ile gösterim:**
  - Her düğümün en fazla kaç çocuğu olacağını bilmek gerekir.
  - Örneğin her düğümün en fazla 3 çocuğu varsa kök 0. göze, çocukları 1, 2, 3. gözlere yerleşir.
  - Seviye seviye ilerlenir ve olmayan çocuklar için boş göz bırakılır.
  - Boş gözler çok yer kaplar. Maksimum çocuk sayısının hiç aşılmayacağından emin olmak gerekir.
- **Bağlı liste ile gösterim:**
  - Her düğümde en fazla çocuk sayısı kadar işaretçi bulunur. Olmayan çocuklar `NULL` gösterir.
  - Ekleme ve silme daha kolaydır. Bir düğüm silinince ebeveyninin işaretçisi güncellenir.
  - Ağaçlarda genellikle bu yöntem kullanılır.

İkili Ağaç

- Her düğümün en fazla iki çocuğu vardır.
- Yapı: kök, sol alt ağaç (left subtree) ve sağ alt ağaç (right subtree).

- **Dizi ile gösterim:** i. düğümün sol çocuğu $2i + 1$, sağ çocuğu $2i + 2$. gözdedir.

Derste kullanılan örnek (ekrandan, 02:20:35):

```
        A(0)
       /    \
   B(1)      C(2)
           /    \
       D(5)     E(6)
                   \
                   F(14)
                      \
                      G(30)
```

İndis	0	1	2	3	4	5	6	...	14	...	30
Değer	A	B	C	boş	boş	D	E	...	F	...	G

- 7 düğüm için 31 göz gerekir. Boş yer problemi büyüktür.
- Bu yüzden ikili ağaç genellikle bağlı yapıyla gösterilir. Her düğümde değer, sol adres ve sağ adres bulunur.

Ağaçta Gezinme

- Ağacın tamamında işlem yapmak için düğümler belli bir sırayla gezilir.
- Aritmetik ifade ağacı (expression tree) örneği `A + B`:
  - Preorder (düğüm, sol, sağ): `+ A B`. Ön ek (prefix) gösterimi.
  - Inorder (sol, düğüm, sağ): `A + B`. Ara ek (infix) gösterimi.
  - Postorder (sol, sağ, düğüm): `A B +`. Son ek (postfix) gösterimi.
- `A + B * C` ifade ağacı: kök `+`, sol çocuk `A`, sağ çocuk `*`. `*` düğümünün çocukları `B` ve `C`'dir.

Gezinme	Sonuç
Preorder	`+ A * B C`
Inorder	`A + B * C`
Postorder	`A B C * +`

- "Sol" denildiğinde sol alt ağacın tamamı bitirilir. Sonra sağ alt ağaca geçilir.
- Alt ağaç da bir ağaç olduğu için problem özyinelemeli olarak çözülür.

`printPreorder` (ekrandan, 02:34:25):

```c
typedef struct node
{
    int value;
    struct node *left, *right;
} NODE;

void printPreorder(NODE *treeNode)
{
    if (treeNode != NULL)
    {
        printf("%d\n", treeNode->value);
        printPreorder(treeNode->left);
        printPreorder(treeNode->right);
    }
}
```

- Düğüm `NULL` değilse önce değeri yazılır.
- Sonra fonksiyon sol çocuk için çağrılır. Böylece sol taraf baştan sona bitirilir.
- `NULL`'a gelince fonksiyondan çıkılır ve sağ çocuk çağrısına geçilir.
- Hocanın önerisi:

  - Elle küçük bir ifade ağacı kurun.
  - Preorder yazdırıp prefix gösterimini görün.
  - `printInorder` ve `printPostorder` fonksiyonlarını kendiniz yazın.
  - Ağaca ekleme ve silme bir sonraki derste anlatılacak.

Yığın Alıştırması

- Hoca bir soruyu konuştu. Girdi `A, B, C, 2, D, E, 1` gibi harf ve rakamlardan oluşur.
- Harfler yığına itilir (push). Rakam gelince o kadar eleman yığından çekilir (pop).
- `A, B, C` itilir. `2` ile iki eleman çekilir, yığında A kalır.
- `D, E` itilir. `1` ile bir eleman çekilir, yığında A ve D kalır.
- Soru string ile de çözülebilir. Ama karmaşıklığın $O(n)$'i aşmaması gerekir. Diziyi sürekli ileri geri taramak bunu aşar.

Ders sonrası notlar

- `createQueue` ve `newNode` kodlarındaki `malloc` dönüş değeri kontrol edilmemiştir. Hoca bunu bilerek yazmadığını söyledi. Gerçek kodda `NULL` kontrolü yapılmalıdır.
- Derste yazılan kuyruk kodu, `isEmpty` eklenerek ajan tarafından `gcc -Wall` ile derlendi ve küçük bir `main` ile denendi. `isEmpty` derste yazılmamıştır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.